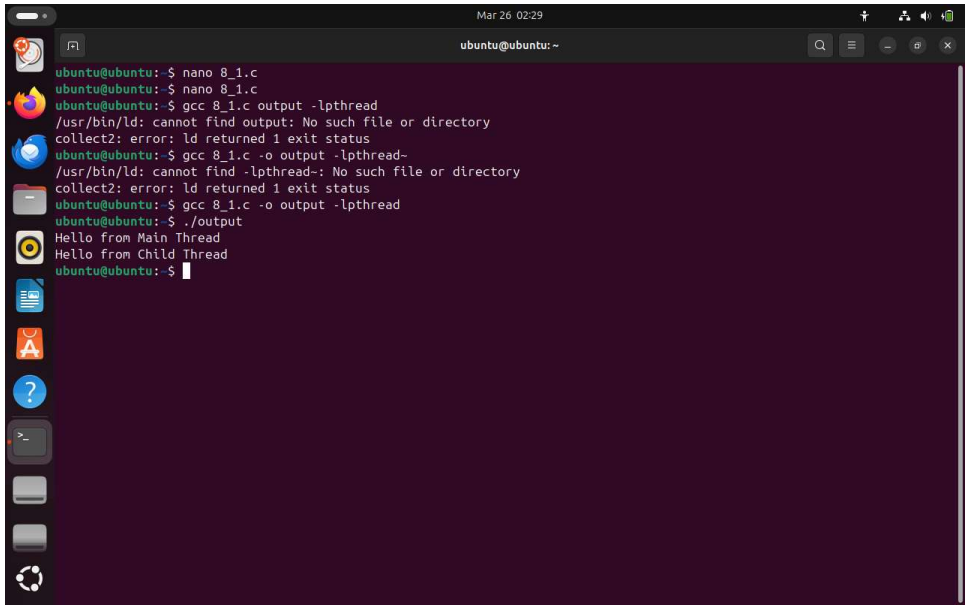
NEHA SHARMA - U24AI050 LAB-8

26 March 2026 07:15

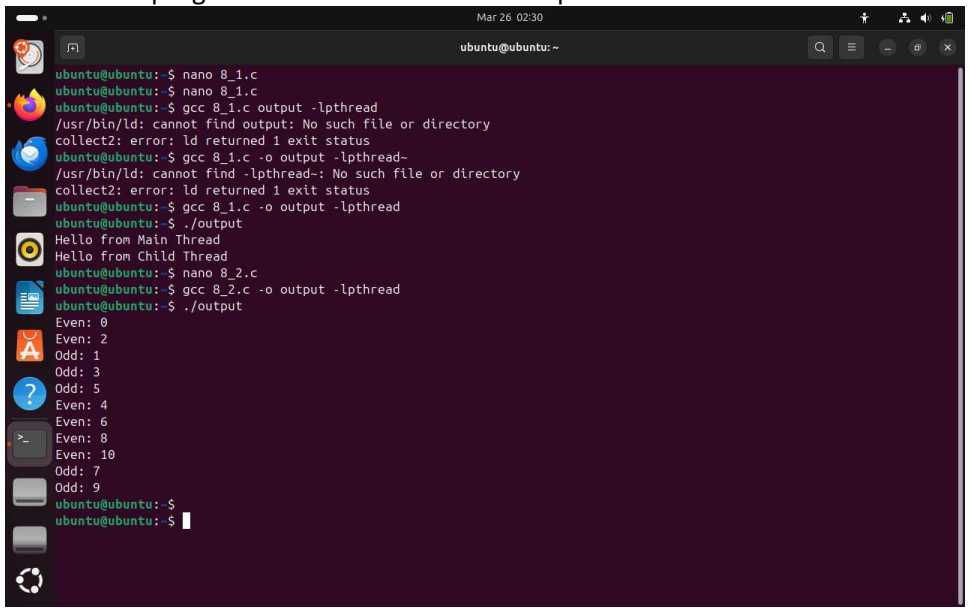
1. Write a C program to create a single thread and print messages from both main and child thread.




```
#include <stdio.h>
#include <pthread.h>

void* thread_func(void* arg) {
    printf("Hello from Child Thread\n");
    return NULL;
}

int main() {
    pthread_t t1;

    pthread_create(&t1, NULL, thread_func, NULL);

    printf("Hello from Main Thread\n");

    pthread_join(t1, NULL);
```

```
    return 0;
}
```

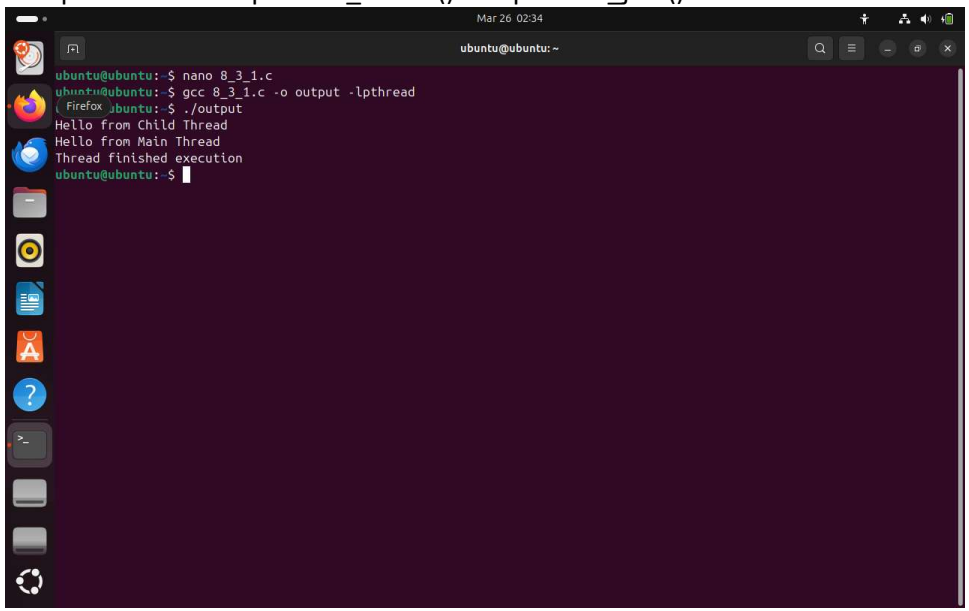
2. Write a C program to create two threads to print odd and even numbers.




```
#include <stdio.h>
#include <pthread.h>

void* even(void* arg) {
    for(int i=0; i<=10; i+=2)
        printf("Even: %d\n", i);
    return NULL;
}

void* odd(void* arg) {
    for(int i=1; i<=10; i+=2)
        printf("Odd: %d\n", i);
    return NULL;
}

int main() {
    pthread_t t1, t2;
```

```
    pthread_create(&t1, NULL, even, NULL);
    pthread_create(&t2, NULL, odd, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

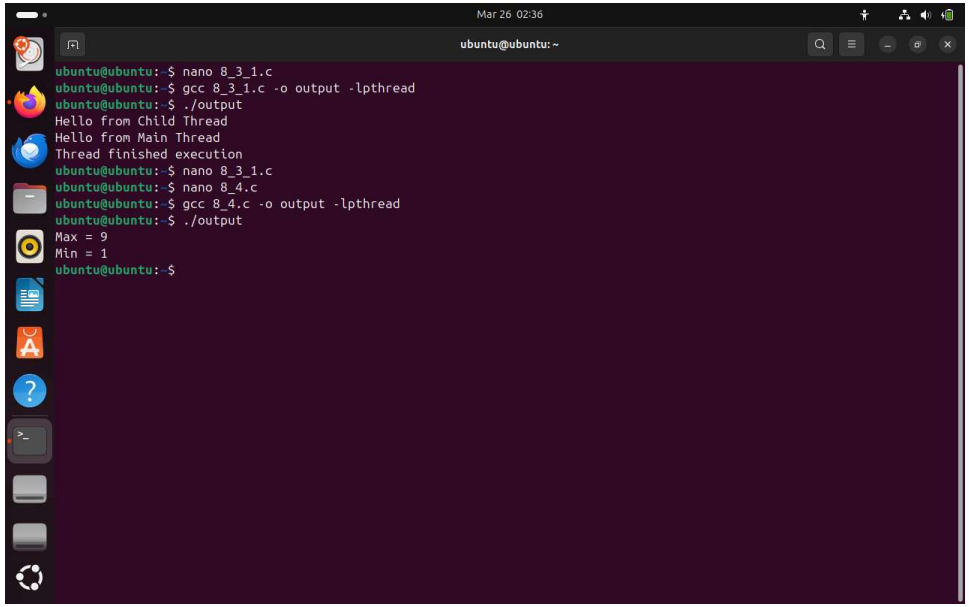
3. Explain the use of pthread_create() and pthread_join().




```c
#include <stdio.h>
#include <pthread.h>

// Thread function
void* myThread(void* arg) {
    printf("Hello from Child Thread\n");
    return NULL;
}

int main() {
    pthread_t thread_id;

    // Thread create
    pthread_create(&thread_id, NULL, myThread, NULL);

    printf("Hello from Main Thread\n");

    // Thread join (wait karega)
    pthread_join(thread_id, NULL);

    printf("Thread finished execution\n");

    return 0;
```

}

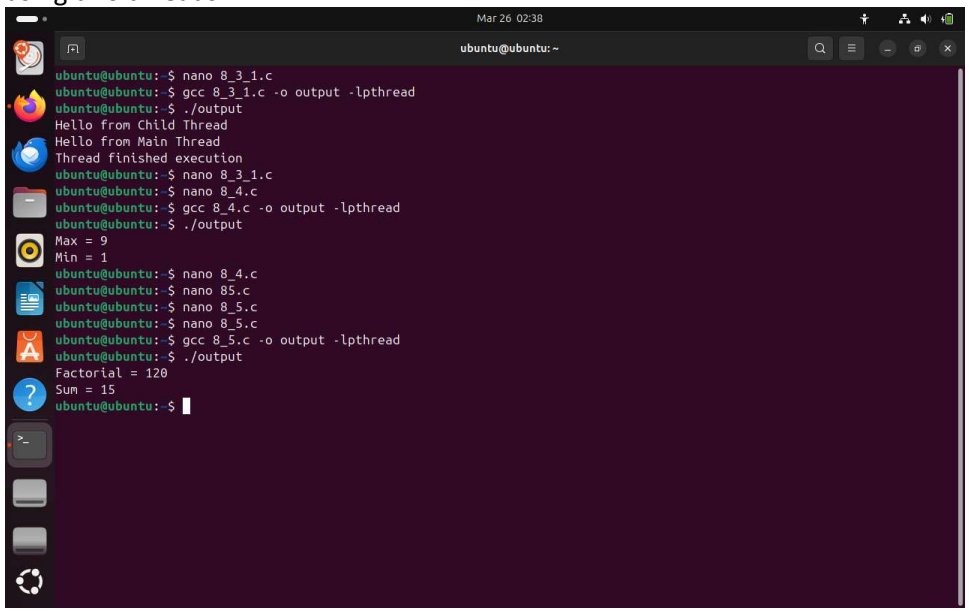
4. Write a multithreaded C program to find maximum and minimum in an array using two threads.

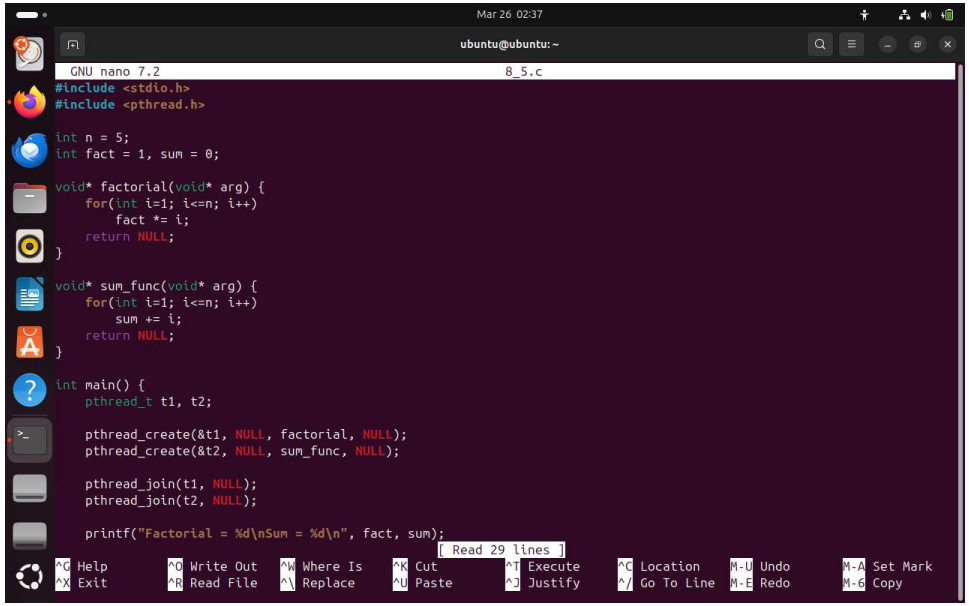



```
#include <stdio.h>
#include <pthread.h>

int arr[] = {5, 2, 9, 1, 7};
int max, min;

void* find_max(void* arg) {
   max = arr[0];
   for(int i=1; i<5; i++)
      if(arr[i] > max) max = arr[i];
   return NULL;
}

void* find_min(void* arg) {
   min = arr[0];
   for(int i=1; i<5; i++)
      if(arr[i] < min) min = arr[i];
   return NULL;
```

```
}

int main() {
   pthread_t t1, t2;

   pthread_create(&t1, NULL, find_max, NULL);
   pthread_create(&t2, NULL, find_min, NULL);

   pthread_join(t1, NULL);
   pthread_join(t2, NULL);

   printf("Max = %d\nMin = %d\n", max, min);
}
```

5. Write a program to compute factorial and sum of first N natural numbers using two threads






```
#include <stdio.h>
#include <pthread.h>

int n = 5;
int fact = 1, sum = 0;

void* factorial(void* arg) {
   for(int i=1; i<=n; i++)
      fact *= i;
   return NULL;
}

void* sum_func(void* arg) {
   for(int i=1; i<=n; i++)
      sum += i;
   return NULL;
}

int main() {
   pthread_t t1, t2;

   pthread_create(&t1, NULL, factorial, NULL);
   pthread_create(&t2, NULL, sum_func, NULL);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Factorial = %d\nSum = %d\n", fact, sum);
}
```

6. Explain the concept of race condition.

Introduction

In multithreading, multiple threads execute simultaneously within a program. When these threads access shared resources (such as variables, memory, or files) without proper control, it may lead to incorrect or unpredictable results. This situation is known as a **Race Condition**.

What is a Race Condition?

A Race Condition occurs when:

- Two or more threads access the same shared data at the same time
- The final result depends on the order of execution of the threads

In simple terms, threads “race” against each other to access shared resources, which can lead to inconsistent outcomes.

Example

Consider a shared variable:

```
int counter = 0;
```

Two threads perform the following operation:

```
counter++;
```

This operation internally consists of three steps:

1. Read the value of counter
2. Increment the value
3. Write the updated value back

If both threads execute simultaneously:

- They may read the same value at the same time
- The incremented value may overwrite each other

Expected result: 2000

Actual result: unpredictable (e.g., 1500 or any other value)

Causes of Race Condition

- Use of shared variables

- Lack of synchronization
- Simultaneous execution of multiple threads
- Unprotected critical sections

Effects of Race Condition

- Incorrect output
- Data inconsistency
- Unpredictable behavior
- Difficult to debug

How to Avoid Race Condition

Race conditions can be avoided using **thread synchronization techniques**, such as:

Methods:

- Mutex (Mutual Exclusion)
- Semaphore
- Locks

Example using Mutex:

```
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);
```

This ensures that only one thread can access the critical section at a time.

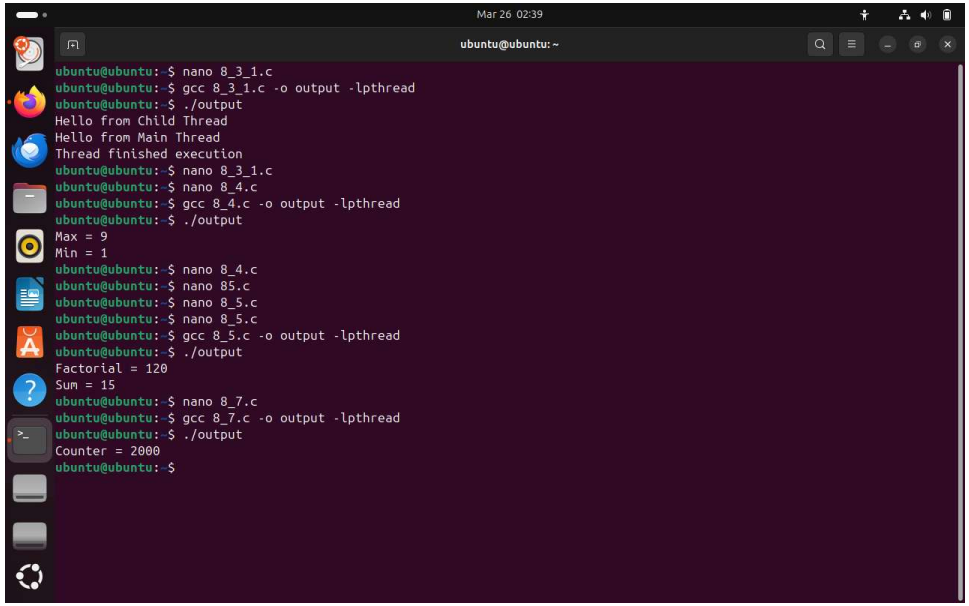
Real-Life Example

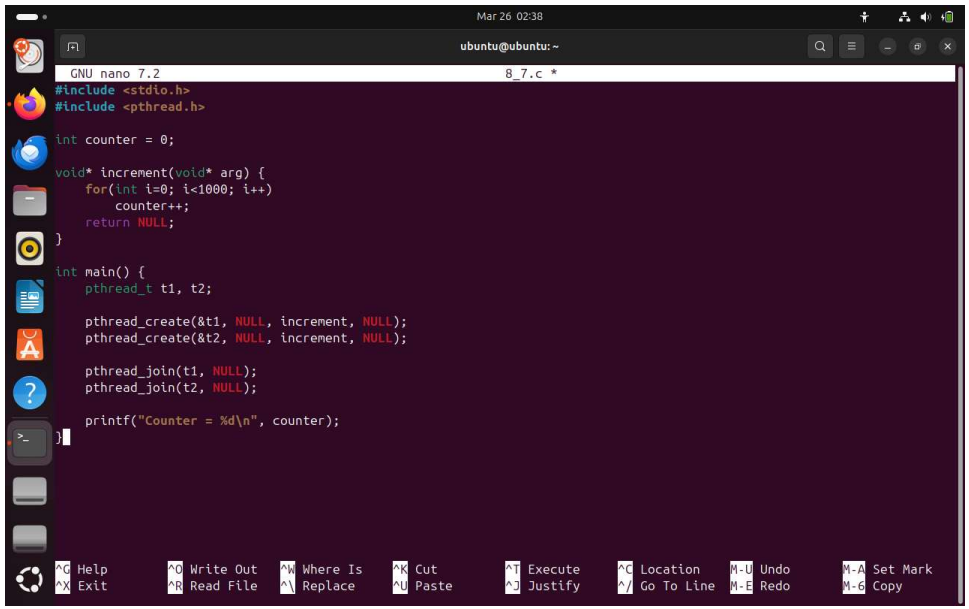
Consider a bank account:
If two people withdraw money from the same account at the same time, the balance may become incorrect. This is an example of a race condition.

Conclusion

A Race Condition is a major issue in multithreading that can lead to incorrect and unpredictable results. It is important to use synchronization techniques like mutexes to protect shared resources and ensure correct program execution.

.7. Write a C program to demonstrate race condition in a shared counter.



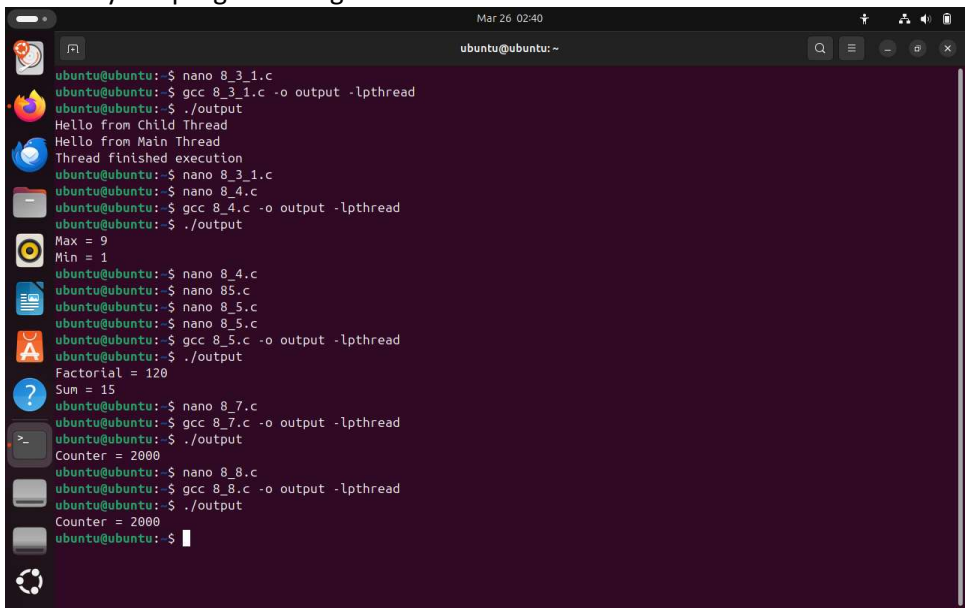


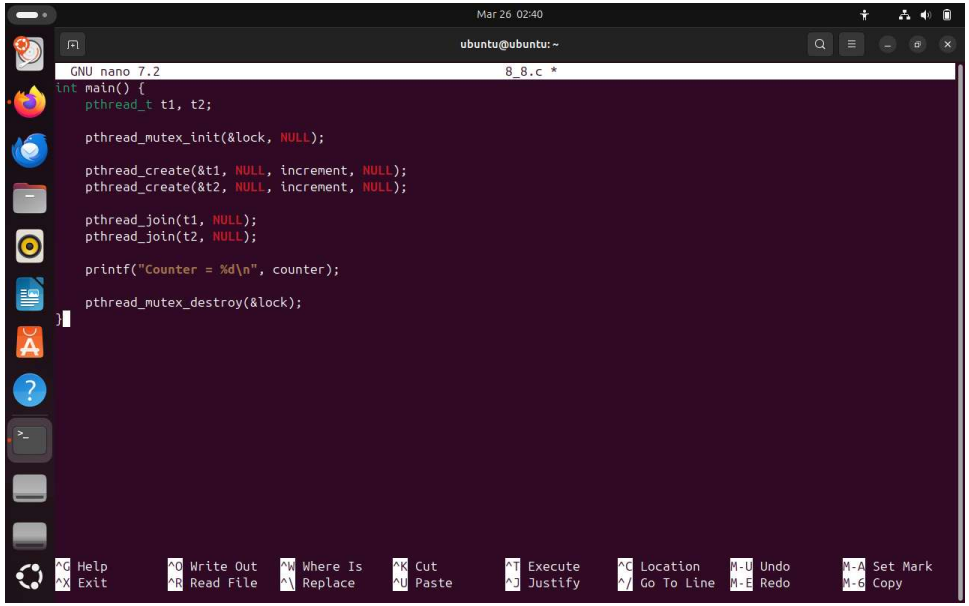
#include <stdio.h>
#include <pthread.h>

int counter = 0;

```
void* increment(void* arg) {
   for(int i=0; i<1000; i++)
      counter++;
   return NULL;
}

int main() {
   pthread_t t1, t2;

   pthread_create(&t1, NULL, increment, NULL);
   pthread_create(&t2, NULL, increment, NULL);

   pthread_join(t1, NULL);
   pthread_join(t2, NULL);

   printf("Counter = %d\n", counter);
}
```

8. Modify the program using mutex to avoid race condition.

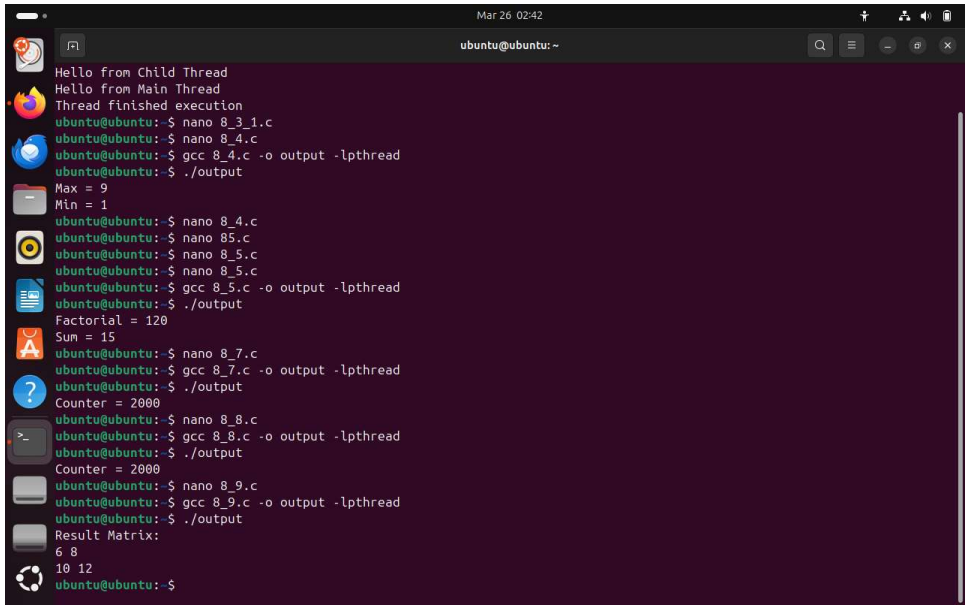


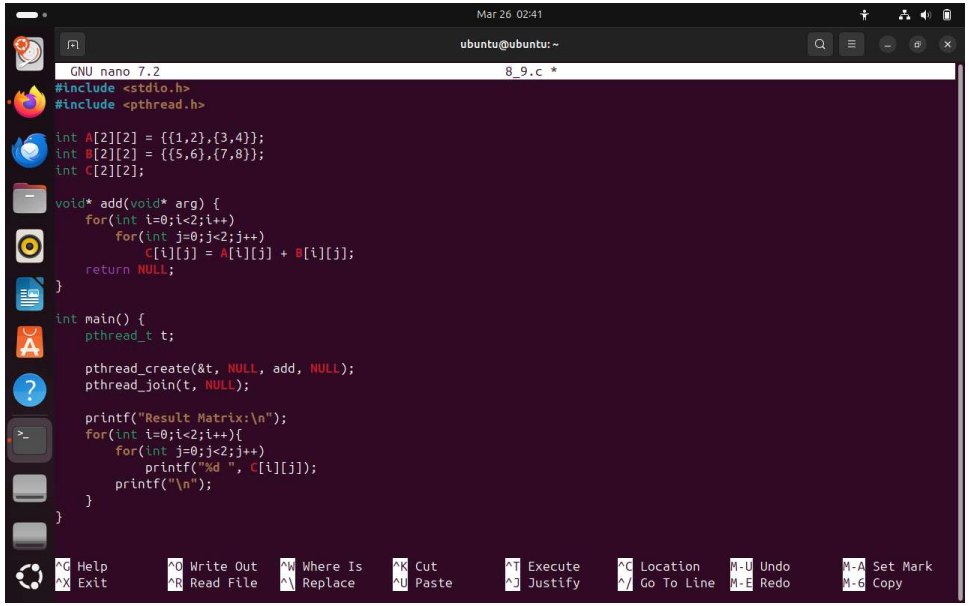



```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void* increment(void* arg) {
   for(int i=0; i<1000; i++) {
      pthread_mutex_lock(&lock);
      counter++;
      pthread_mutex_unlock(&lock);
   }
   return NULL;
}

int main() {
   pthread_t t1, t2;

   pthread_mutex_init(&lock, NULL);

   pthread_create(&t1, NULL, increment, NULL);
   pthread_create(&t2, NULL, increment, NULL);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter = %d\n", counter);

    pthread_mutex_destroy(&lock);
}
```

9. Write a multithreaded program for matrix addition or matrix multiplication.\






```
#include <stdio.h>
#include <pthread.h>

int A[2][2] = {{1,2},{3,4}};
int B[2][2] = {{5,6},{7,8}};
int C[2][2];

void* add(void* arg) {
   for(int i=0;i<2;i++)
      for(int j=0;j<2;j++)
         C[i][j] = A[i][j] + B[i][j];
   return NULL;
}

int main() {
   pthread_t t;

   pthread_create(&t, NULL, add, NULL);
   pthread_join(t, NULL);

   printf("Result Matrix:\n");
   for(int i=0;i<2;i++){
      for(int j=0;j<2;j++)
```

```
            printf("%d ", C[i][j]);
        printf("\n");
    }
}
```

10. Explain thread synchronization and critical section.

Introduction

In multithreading, multiple threads execute concurrently within the same program. When these threads access shared resources such as variables, memory, or files, conflicts may occur. To avoid such issues and ensure correct execution, **Thread Synchronization** is used.

Critical Section

A **Critical Section** is the part of a program where shared resources are accessed.

- Only one thread should execute the critical section at a time
- If multiple threads enter it simultaneously, it may lead to incorrect results

Example

```
counter++;   // Critical Section
```

If multiple threads update counter at the same time, the result may become inconsistent.

Thread Synchronization

Thread Synchronization is a technique used to control the execution of multiple threads so that shared resources are accessed safely.
It ensures:

- Data consistency
- Proper execution order
- Prevention of conflicts

Need for Synchronization

Synchronization is required because:

- Threads run concurrently
- Shared data may be modified simultaneously
- It prevents race conditions

Methods of Thread Synchronization

1. Mutex (Mutual Exclusion)

- A mutex is a locking mechanism
- Only one thread can access the critical section at a time

```
pthread_mutex_lock(&lock);
counter++;
pthread_mutex_unlock(&lock);
```

2. Semaphore

- Allows a limited number of threads to access a resource
- Uses a counter to control access

3. Locks

- General mechanism to protect shared resources
- Ensures mutual exclusion

Working of Synchronization

1. A thread requests access to the critical section
2. It acquires the lock
3. Executes the critical section
4. Releases the lock
5. Another thread can now enter

Advantages of Thread Synchronization

- Ensures correct output
- Maintains data consistency
- Prevents race conditions
- Provides safe access to shared resources

Disadvantages

- Can reduce performance due to waiting
- May lead to deadlock if not handled properly
- Increases program complexity

Conclusion

Thread Synchronization is essential in multithreaded programs to ensure safe and correct access to shared resources. The critical section must

be protected using mechanisms like mutexes and semaphores to avoid issues such as race conditions and ensure reliable program execution